

- The assignment is due at Gradescope on 5/2/25.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using $\text{\LaTeX}$. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work*. Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.
- Each student can request a total of up to 5 days of extensions on all homework - either for original submission or resubmission. Up to two days can be requested for a homework. If you would like to request an extension, please fill out the [Homework extension form](#). Extensions do not carry any late penalty. If there is a life emergency that might prevent you from submitting on time not covered by this policy, email the Head TA.
- Taking into account the different focus students have coming into this course, you can choose between a more advanced theory problem or a programming assignment. **Complete either theory problem 4 or programming problem 5.**
- Is anyone still reading these?

PROBLEM 1 (DRIVING DOWN MEMORY LANE) Lorenzo wants to drive from Boston to Chicago, taking the shortest walk he can. However, he also really wants to drive along a route that passes through Princeton University to visit some old colleagues. Find the shortest walk that passes through the university.

More formally, you are given a connected, directed, weighted graph $G = (V, E)$, where each $e \in E$ represents a different route, $w(e)$ is the length of the route e , and each $v \in V$ represents a point connecting some number of different routes. Furthermore, $s \in V$ represents Boston, and $t \in V$ represents Chicago, and there is some edge $(y_1, y_2) \in E$ representing the edge through Princeton University that Lorenzo must take. Note that the edge (y_2, y_1) may exist, but is not the route that Lorenzo wants to take. Note also that it is possible that taking this edge forces Lorenzo to make a cycle - this is okay.

- (a) Provide pseudocode for an algorithm that finds the desired walk by running Dijkstra's a constant number of times. Sketch a proof of correctness. You do not need to analyze runtime, but it should not take longer than $O((|V| + |E|) \log |V|)$ (assuming a Binary heap implementation of Dijkstra's).
- (b) Describe (in words) a way to modify G to make $G' = (V', E')$, such that running Dijkstra's algorithm (unmodified) a single time on G' lets you find the correct walk with a linear amount of processing. Your modifications should not affect the algorithm's total asymptotic runtime. Sketch a proof of correctness, and sketch a proof that the modifications do not affect the total runtime.

Collaborators:

Solution:

PROBLEM 2 (COLORFUL MST) You are given access to a program P to find the MST of a weighted undirected graph with any edge weights. However you are now given as input a weighted undirected graph and some edges are colored red while the others are blue. You further know that all the edge weights in the input given to you are integers. Design an algorithm that takes only $O(|V| + |E|)$ additional time, **apart from the time taken in a single call to P** , to output a MST of your input graph that uses as few red edges as possible. Note that **you still want to output a minimum cost spanning tree**; so the goal is to minimize the total weight, but of the many possible spanning trees that minimize the weight, you want one that uses as few red edges as possible. Prove the correctness of your algorithm and justify its runtime.

Collaborators:

Solution:

PROBLEM 3 (HUFFMAN CODING) *A geneticist wants to encode a genetic sequence, and plans to use Huffman Coding to do so.*

- (a) Say the nucleotides A, C, G, T appear in the sequence with frequencies 31%, 20%, 9%, 40% respectively. What is the Huffman encoding of these four characters?*
- (b) The geneticist is an alien geneticist, and these extraterrestrial species have n different nucleotides in their genes. If the geneticist encoded their sequences, what is the longest possible code word? Provide a set of frequencies that achieves this length.*
- (c) Find the largest value p such that, if all nucleotide frequencies are less than $p\%$, codewords are guaranteed to not have length 1. Sketch a proof.*

Collaborators:

Solution:

You can choose to complete this theory problem or the programming assignment

PROBLEM 4 (DYNAMIC MINIMUM SPANNING TREES) We are given an undirected connected weighted graph $G = (V, E, w)$, and a minimum spanning tree T of G with respect to these weights. We want to update T as vertices and edges are added to or deleted from G without recomputing T from scratch.

- (a) Let $G' = G \cup \{e\}$ for some new edge e with weight $w(e)$. Write **pseudocode** for an algorithm which takes in G , T , and e , and returns an MST of G' . Your algorithm must run in $O(V)$ time. **Prove your algorithm is correct**, and analyze its running time.
- (b) Let $G' = G \setminus \{e\}$ for some $e \in G$. Write **pseudocode** for an algorithm which takes in G , T , and e , and returns an MST of G' . You may assume G' is still connected. Your algorithm must run in $O(E)$ time. **Prove your algorithm is correct**, and analyze its running time.
- (c) Let $G' = G \setminus \{v, e_1, \dots, e_k\}$ where $v \in G$ and $e_1, e_2, \dots, e_k$ are adjacent to v . Prove that T may provide no information about any MST T' of G' . Specifically, give an example of $G = (V, E, w)$ and $G' = G - v$ such that the MST of G has no edges in common with the MST of G' .
- (d) Let $G' = G \cup \{v, e_1, e_2, \dots, e_k\}$ where v is a new vertex and edges $e_1, e_2, \dots, e_k$ are incident to v . Let $w(e_i)$ define the weight of new edge i . Write **pseudocode** for an algorithm which takes in G , T , v , e_i , and returns an MST of G' . You may assume G' is still connected. The running time of your algorithm should not depend on $|E|$, the number of edges in G . **Prove your algorithm is correct**, and analyze its running time. **No credit for $\Omega(E \log V)$ time algorithms.**

Collaborators:

Solution:

You can choose to complete this programming problem or the previous theory assignment

PROBLEM 5 (EV DOMINATION) In the future, all cars are electric. Moreover, for long journeys, instead of charging your car, you can just grab a new battery from one of the charging stations to replace the old one. Because of different voltages and capacities of different batteries, each battery can make the car go at different speeds until they run out. Lorenzo is going on another trip, this time from Chicago to San Francisco, and needs your help to decide how he can get there the fastest. Lorenzo starts in Chicago (vertex 1) and needs to decide at which charging stations he needs to change batteries to get to San Francisco (vertex N). Formally, you are given a directed graph with N vertices, each of which has a battery that can power the car up to **distance** c_i with **speed** s_i , and M edges (roads) each of which has length $d(u, v)$. [Testcases can be found here](#).

Input: In the first line you are given two integers N , and M . In each of the next N lines there are two integers: c_i and s_i . M lines follow, each with three integers: u , v , and $d(u, v)$.

Output: Your program should output a single floating point number with **exactly** six points of precision, the least time it takes Lorenzo to go from vertex 1 to vertex N .

Limits:

$$N \leq 200$$

$$M \leq N^2$$

$$T \leq 1s$$

Sample Input

```
3 2
2 3
2 4
4 4
1 2 1
2 3 1
```

Sample Output

```
0.583333 (1/3 + 1/4)
```